

Module 12 - chapter 4

Resources & Data Sources

Resources

We use keyword -> resource

And we need to know the name of the resource, for example we want to create a VPC

<provider>_<resourceType>

```
1 provider "aws" {
2   region = "eu-central-1"
3   access_key = "AKIAUZ7NN5BBID2IQDHY"
4   secret_key = "HCVKNGYvYzKU5hSs4/pwkM2KWFC5hGRKAG/DaniW"
5 }
6
7 resource "aws_"
```

<provider> _<resourceType>

name of the resource -> "aws_vpc"

The screenshot shows the Terraform documentation for the `aws_vpc` resource. On the left, there's a search bar with 'vpc' entered, showing 179 matching results. The `aws_vpc` resource is highlighted in the list. The main content area is titled 'Resource: aws_vpc' and describes it as 'Provides a VPC resource.' It includes a 'NOTE' about AWS GuardDuty and a 'Basic usage' section with a code snippet: `resource "aws_vpc" "main" { cidr_block = "10.0.0.0/16" }`. There are also links for 'Example Usage', 'Argument Reference', and 'Attribute Reference'.

Name inside terraform, a **variable name** that we define -> "development-vpc"
see line 7 below:

```
main.tf x providers.tf x .terraform.lock.hcl x
1 provider "aws" {
2     region = "eu-west-2"
3     access_key = "AKIA2JGYSYB2DWZEUS47"
4     secret_key = "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"
5 }
6
7 resource "aws_vpc" "development-vpc" {
8     | You, 3 minutes ago • Uncommitted changes
9 }
```

And inside the block we can pass in parameters

```
7 resource "aws_vpc" "development-vpc" {
8
9 }
```

Each resource block describes one or more infrastructure objects

VPC needs an infrastructure object called CIDR block to define the IP address range for our VPC:

```
6
7 resource "aws_vpc" "development-vpc" {
8     cidr_block = "10.0.0.0/16" You, A minute a
9 }
```

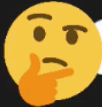
Let's add another resource, a subnet:

```
7 resource "aws_vpc" "development-vpc" {
8     cidr_block = "10.0.0.0/16"
9 }
10
11 resource "aws_subnet" "dev-subnet-1" {
12
13 }
```

You, Moments ago • Uncommitted changes

Now a subnet is part of a VPC so we need to define the VPC that the subnet will be part of. And we want the subnet to be part of VPC "development-vpc" but this VPC resource doesn't exist just yet:

```
11 resource "aws_subnet" "dev-subnet-1" {
12     vpc_id =
13 }
```

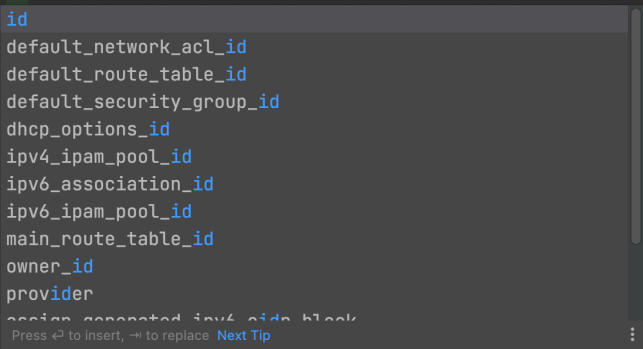
 Creating a resource for a non existing resource

So we reference the resource to get the object-> resource name.varName

-> aws_vpc.development-vpc

And now from the object we can access the object's attributes like ID:

```
10
11 resource "aws_subnet" "dev-subnet-1" {
12   vpc_id = aws_vpc.development-vpc.id  You, Moments ago • Uncommitted changes
13 }
```



So we have referenced a resource that hasn't been created yet.

We can now define the CIDR block for the subnet:

```
10
11 resource "aws_subnet" "dev-subnet-1" {
12   vpc_id = aws_vpc.development-vpc.id
13   cidr_block = "10.0.10.0/24"  You, 3 minutes
14 }
```

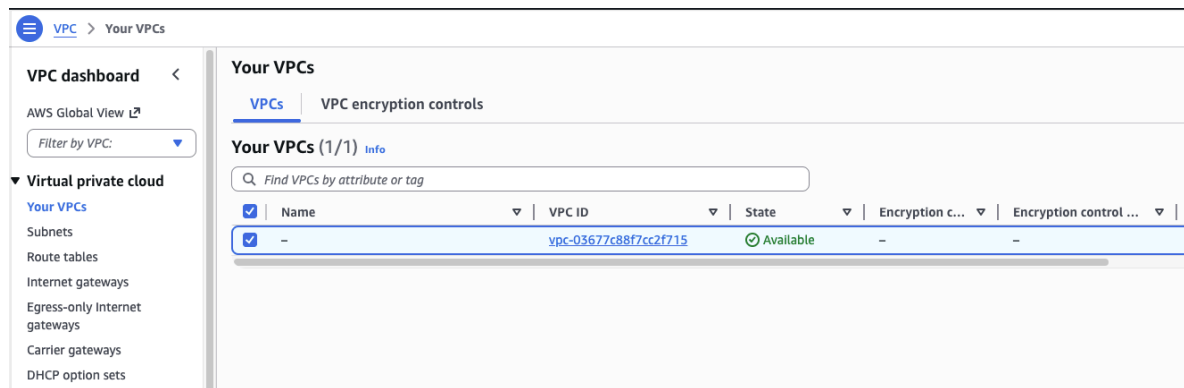
And we define the AZ the subnet will be created in, at the moment it will use a default AZ but we want to specify it:

```
10
11 resource "aws_subnet" "dev-subnet-1" {
12   vpc_id = aws_vpc.development-vpc.id
13   cidr_block = "10.0.10.0/24"
14   availability_zone = "eu-west-2a"  You, 2 minutes
15 }
```

Terraform apply

terraform apply —> reads your .tf files, figures out what needs to be created/changed/deleted in AWS, shows you a plan, and after you confirm it executes the changes.

Currently my AWS has only one VPC



After I execute terraform apply I will have two VPCs, one of which was created by terraform

-> terraform apply

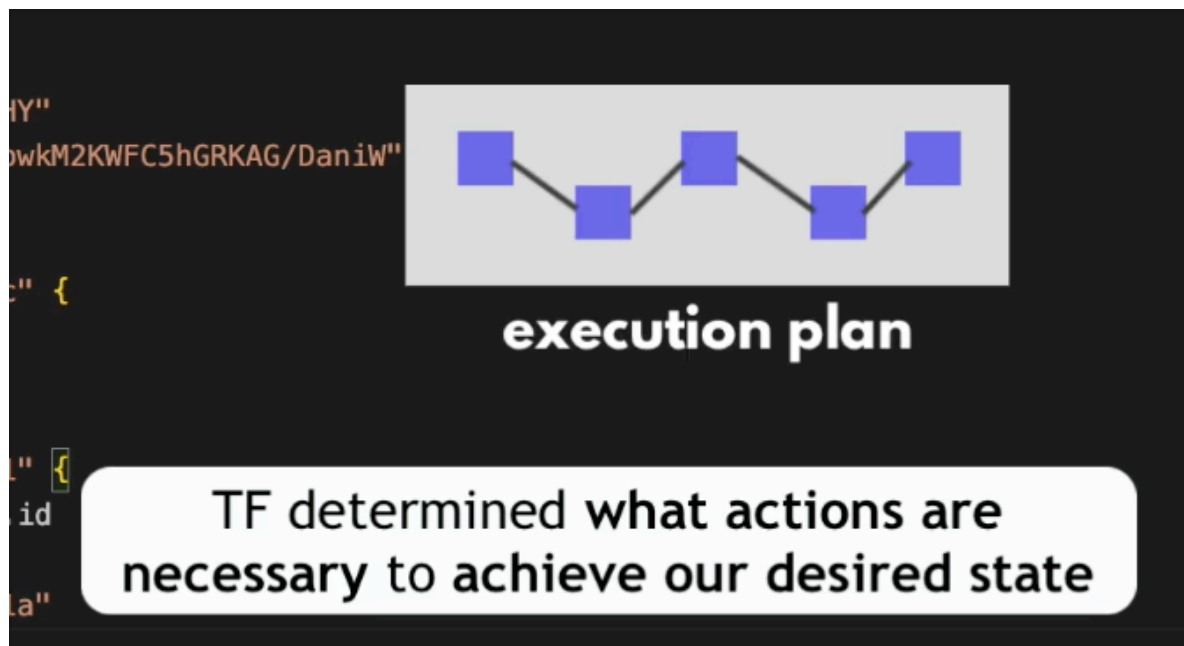
```
➜ terraform git:(Module_12/Terraform_chapter-3) x terraform apply
Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_subnet.dev-subnet-1 will be created
+ resource "aws_subnet" "dev-subnet-1" {
  + arn                               = (known after apply)
  + assign_ipv6_address_on_creation   = false
  + availability_zone                 = "eu-west-2a"
  + availability_zone_id              = (known after apply)
  + cidr_block                        = "10.0.10.0/24"
  + enable_dns64                     = false
  + enable_resource_name_dns_a_record_on_launch = false
  + enable_resource_name_dns_aaaa_record_on_launch = false
  + id                               = (known after apply)
  + ipv6_cidr_block                  = (known after apply)
  + ipv6_cidr_block_association_id   = (known after apply)
  + ipv6_native                      = false
  + map_public_ip_on_launch          = false
  + owner_id                         = (known after apply)
  + private_dns_hostname_type_on_launch = (known after apply)
  + region                           = "eu-west-2"
  + tags_all                         = (known after apply)
  + vpc_id                           = (known after apply)
}

# aws_vpc.development-vpc will be created
+ resource "aws_vpc" "development-vpc" {
  + arn                               = (known after apply)
  + cidr_block                        = "10.0.0.0/16"
  + default_network_acl_id            = (known after apply)
  + default_route_table_id            = (known after apply)
  + default_security_group_id         = (known after apply)
  + dhcp_options_id                   = (known after apply)
  + enable_dns_hostnames               = (known after apply)
  + enable_dns_support                 = true
  + enable_network_address_usage_metrics = (known after apply)
  + id                               = (known after apply)
}
```

Terraform is showing us the plan, the actions that need to be taken to achieve our desired state 🙌



```
+ instance_tenancy      = "default"  
+ ipv6_association_id  = (known after apply)  
+ ipv6_cidr_block       = (known after apply)  
+ ipv6_cidr_block_network_border_group = (known after apply)  
+ main_route_table_id   = (known after apply)  
+ owner_id              = (known after apply)  
+ region                = "eu-west-2"  
+ tags_all              = (known after apply)  
}
```

Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: █

We also can see the summary

```
+ region                = "eu-west-2"  
+ tags_all              = (known after apply)  
}
```

Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

And we have to provide a value

We need to **confirm** the action Terraform will take

```

+ region          = "eu-west-2"
+ tags_all        = (known after apply)
}

```

Plan: 2 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value:

If after reviewing the information everything looks as we want to, then we type 'yes' any other string will result in canoeing the 'terraform apply'

-> yes

Do you want to perform these actions?

Terraform will perform the actions described above.

Only 'yes' will be accepted to approve.

Enter a value: yes

aws_vpc.development-vpc: Creating...

aws_vpc.development-vpc: Creation complete after 2s [id=vpc-0e43e232c4a94ae2a]

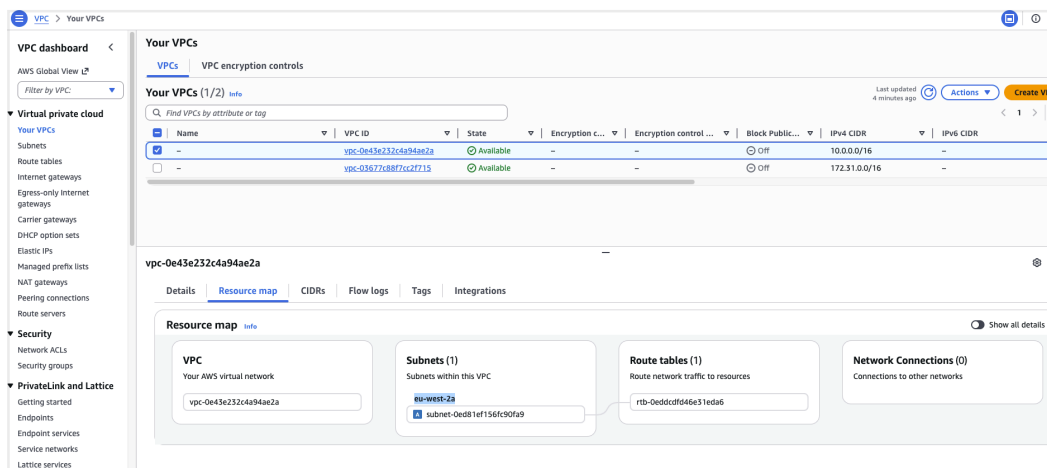
aws_subnet.dev-subnet-1: Creating...

aws_subnet.dev-subnet-1: Creation complete after 0s [id=subnet-0ed81ef156fc90fa9]

Apply complete! Resources: 2 added, 0 changed, 0 destroyed.

→ terraform git:(Module_12/Terraform_chapter-3) ✖

And in my AWS management console I can see my new VPC:



The VPC doesn't have a name because we didn't set a tag in tf for the vpc.

And here is the new subnet

The screenshot shows the AWS Global View console. On the left, there's a navigation menu with categories like 'Virtual private cloud', 'Security', and 'PrivateLink and Lattice'. The main area displays a table of subnets. The first row is selected, showing details for 'subnet-0ed81ef156fc90fa9'.

Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 CIDR	IPv6 CIDR asso
-	subnet-0ed81ef156fc90fa9	Available	vpc-0e43e232c4a94ae2a	Off	10.0.10.0/24	-	-

subnet-0ed81ef156fc90fa9

Details | Flow logs | Route table | Network ACL | CIDR reservations | Sharing | Tags

Subnet ID subnet-0ed81ef156fc90fa9 IPv4 CIDR 10.0.10.0/24 Availability Zone euw2-az2 (eu-west-2a) Network ACL acl-046b9715e98008bde Auto-assign customer-owned IPv4 address No IPv6 CIDR reservations - Resource name DNS AAAA record Disabled	Subnet ARN arn:aws:ec2:eu-west-2:706976333940:subnet/subnet-0ed81ef156fc90fa9 Available IPv4 addresses 251 Network border group eu-west-2 Default subnet No Customer-owned IPv4 pool - IPv6-only No DNS64 Disabled	State Available IPv6 CIDR - VPC vpc-0e43e232c4a94ae2a Auto-assign public IPv4 address No Outpost ID - Hostname type IP name Owner 706976333940	Block Public Access Off IPv6 CIDR association ID - Route table rtb-0eddcdf046e31eda6 Auto-assign IPv6 address No IPv4 CIDR reservations - Resource name DNS A record Disabled
--	--	--	--

The resources were applied successfully 🥳

Data Sources

What if I want to create a subnet for an existing VPC?
I will need the id of the exiting VPC.

To get the VPC's id I can use the UI but is not efficient, so an alternative is to query the information from AWS using the provider

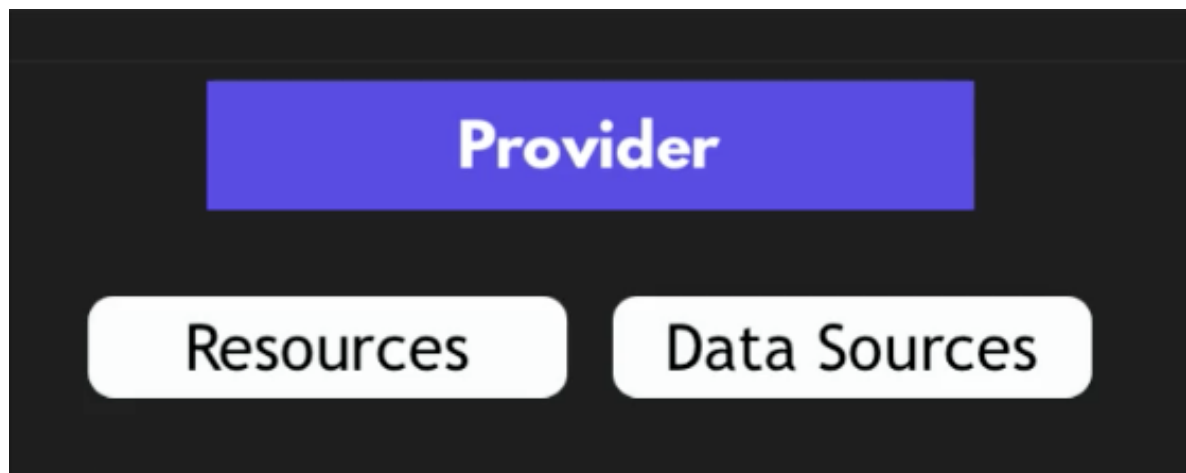
Data Sources

Allow data to be fetched for use
in TF configuration

And the component that the provider give us to query the information is 'data'.

Data -> lets us query the existing resources and components from AWS
('resource' let's us create new resources)

Two types of components that we get from the provider:



We can see the data sources (along with the resources) in the documentation, for example see below the the data sources fro EC2:

Overview [Documentation](#)

AWS DOCUMENTATION

ec2

225 matching results

- aws_placement_group
- aws_spot_datafeed_subscription
- aws_spot_fleet_request
- aws_spot_instance_request

▼ Data Sources

- aws_ami
- aws_ami_ids
- aws_availability_zone
- aws_availability_zones
- aws_ec2_capacity_block_offering

Now let's add a query:

```

17 data "aws_vpc" "existing_vpc" {}
18
19

```

Result of query is exported
under your given name

And in the block we add arguments which are the filter for the query

```

17 data "aws_vpc" "existing_vpc" {}
18     filter = {
19         tags = {

```

arguments = filter for query

Here is the documentation for data source -> aws_vpc

Overview

Documentation

AWS DOCUMENTATION

179 matching results

Data Sources

aws_ec2_managed_prefix_list

aws_ec2_managed_prefix_lists

aws_ec2_network_insights_analysis

aws_ec2_network_insights_path

aws_internet_gateway

aws_nat_gateway

aws_nat_gateways

aws_network_acls

aws_network_interface

aws_network_interfaces

aws_prefix_list

aws_route

aws_route_table

aws_route_tables

aws_security_group

aws_security_groups

aws_subnet

aws_subnets

aws_vpc

aws_vpc_dhcp_options

aws_vpc_endpoint

Data Source: aws_vpc

aws_vpc

 provides details about a specific VPC.

This resource can prove useful when a module accepts a vpc id as an input variable and needs to, for example, determine the CIDR block of that VPC.

Example Usage

The following example shows how one might accept a VPC id as a variable and use this data source to obtain the data necessary to create a subnet within it.

```

variable "vpc_id" {}

data "aws_vpc" "selected" {
  id = var.vpc_id
}

resource "aws_subnet" "example" {
  vpc_id            = data.aws_vpc.selected.id
  availability_zone = "us-west-2a"
  cidr_block        = cidrsubnet(data.aws_vpc.selected.cidr_block, 4, 1)
}

```

Argument Reference

This data source supports the following arguments:

- region

 - (Optional) Region where this resource will be [managed](#). Defaults to the [Default region in the provider configuration](#).

ON THIS PAGE

Example Usage

Argument Reference

Attribute Reference

Timeouts

[Report an issue](#)

And here are the argument references we can use for data source -> aws_vpc

Data Sources

aws_ec2_managed_prefix_list
aws_ec2_managed_prefix_lists
aws_ec2_network_insights_analysis
aws_ec2_network_insights_path
aws_internet_gateway
aws_nat_gateway
aws_nat_gateways
aws_network_acls
aws_network_interface
aws_network_interfaces
aws_prefix_list
aws_route
aws_route_table
aws_route_tables
aws_security_group
aws_security_groups
aws_subnet
aws_subnets
aws_vpc
aws_vpc_dhcp_options
aws_vpc_endpoint
aws_vpc_endpoint_associations

Argument Reference

Timeouts

Report an issue

This data source supports the following arguments:

- `region` - (Optional) Region where this resource will be `managed`. Defaults to the Region set in the [provider configuration](#).
- `cidr_block` - (Optional) Cidr block of the desired VPC.
- `dhcp_options_id` - (Optional) DHCP options id of the desired VPC.
- `default` - (Optional) Boolean constraint on whether the desired VPC is the default VPC for the region.
- `filter` - (Optional) Custom filter block as described below.
- `id` - (Optional) ID of the specific VPC to retrieve.
- `state` - (Optional) Current state of the desired VPC. Can be either `"pending"` or `"available"`.
- `tags` - (Optional) Map of tags, each pair of which must exactly match a pair on the desired VPC.

More complex filters can be expressed using one or more `filter` sub-blocks, which take the following arguments:

- `name` - (Required) Name of the field to filter by, as defined by [the underlying AWS API](#).
- `values` - (Required) Set of values that are accepted for the given field. A VPC will be selected if any one of the given values matches.

We want terraform to give us an AWS default VPC

```

16
17 data "aws_vpc" "existing_vpc" {
18     default = true
19 }

```

And we want to create a subnet in that existing VPC, but this new resource will reference the 'data source' object instead of a 'resource' object:

```

16
17 data "aws_vpc" "existing_vpc" {
18     default = true
19 }
20
21 resource "aws_subnet" "dev-subnet-2" {
22     vpc_id = data.aws_vpc.existing_vpc.id
23     cidr_block = "10.0.10.0/24"
24     availability_zone = "eu-west-2a"
25 }

```

The subnet name is 'dev-subnet-2' because each name must be unique for each resource type.

Name must be unique for
each resource type

The code needs another change, the CIDR block must be a section of the default VPC's IP address range: 172.31.0.0/16

The screenshot shows the AWS VPC console with a table of VPCs. The table has columns for Name, VPC ID, State, Encryption controls, Block Public IP, IPv4 CIDR, and IPv6 CIDR. Two VPCs are listed: one with CIDR 10.0.0.0/16 and another with CIDR 172.31.0.0/16.

Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...	IPv4 CIDR	IPv6 CIDR
-	vpc-0e43e232c4a94ae2a	Available	-	-	Off	10.0.0.0/16	-
-	vpc-03677c88f7c2f715	Available	-	-	Off	172.31.0.0/16	-

And the default VPC already have subnets that were created by default:

VPC dashboard < Subnets

Find subnets by attribute or tag

Filter by VPC:

Name	Subnet ID	State	VPC	Block Public...	IPv4 CIDR	IPv6 CIDR	IPv6 CIDR
subnet-03da2ce00b3dba523	subnet-03da2ce00b3dba523	Available	vpc-03677c88f7cc2f715	Off	172.31.0.0/20	-	-
subnet-092c85ea013c8c240	subnet-092c85ea013c8c240	Available	vpc-03677c88f7cc2f715	Off	172.31.16.0/20	-	-
subnet-051c96240932f9c7e	subnet-051c96240932f9c7e	Available	vpc-03677c88f7cc2f715	Off	172.31.32.0/20	-	-

subnet-03da2ce00b3dba523

Details | Flow logs | Route table | Network ACL | CIDR reservations | Sharing | Tags

Details	Subnet ARN	State	Block Public Access
Subnet ID subnet-03da2ce00b3dba523	arn:aws:ec2:eu-west-2:706976333940:subnet/subnet-03da2ce00b3dba523	Available	Off
IPv4 CIDR 172.31.0.0/20	Available IPv4 addresses 4090	IPv6 CIDR -	IPv6 CIDR association ID -
Availability Zone euw2-az1 (eu-west-2c)	Network border group eu-west-2	VPC vpc-03677c88f7cc2f715	Route table rtb-0118aea5c82fb9ff
Network ACL acl-01e75d0be99d2ef3f	Default subnet Yes	Auto-assign public IPv4 address Yes	Auto-assign IPv6 address No
Auto-assign customer-owned IPv4 address No	Customer-owned IPv4 pool -	Outpost ID -	IPv4 CIDR reservations -
IPv6 CIDR reservations -	IPv6-only No	Hostname type IP name	Resource name DNS A record Disabled
Resource name DNS AAAA record		Owner	

And we need each subnet to have a different set of IP addresses. So we need to choose a range that is not assigned to the existing subnets.

So I will take the last range: 172.31.32.0/20 and from that we take the next subset -> 172.31.48.0/20

```

16
17 data "aws_vpc" "existing_vpc" {
18     default = true
19 }
20
21 resource "aws_subnet" "dev-subnet-2" {
22     vpc_id = data.aws_vpc.existing_vpc.id
23     cidr_block = "172.31.48.0/20"
24     availability_zone = "eu-west-2a"
25 }

```

-> terraform apply

```

data.aws_vpc.existing_vpc: Read complete after 0s [id=vpc-03677c88f7cc2f715]
aws_subnet.dev-subnet-1: Refreshing state... [id=subnet-0ed81ef156fc90fa9]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# aws_subnet.dev-subnet-2 will be created
+ resource "aws_subnet" "dev-subnet-2" {
  + arn                               = (known after apply)
  + assign_ipv6_address_on_creation   = false
  + availability_zone                 = "eu-west-2a"
  + availability_zone_id              = (known after apply)
  + cidr_block                        = "172.31.48.0/20"
  + enable_dns64                     = false
  + enable_resource_name_dns_a_record_on_launch = false
  + enable_resource_name_dns_aaaa_record_on_launch = false
  + id                               = (known after apply)
  + ipv6_cidr_block                   = (known after apply)
  + ipv6_cidr_block_association_id    = (known after apply)
  + ipv6_native                       = false
  + map_public_ip_on_launch          = false
  + owner_id                         = (known after apply)
  + private_dns_hostname_type_on_launch = (known after apply)
  + region                           = "eu-west-2"
  + tags_all                         = (known after apply)
  + vpc_id                           = "vpc-03677c88f7cc2f715"
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: █

```

And tf calculates the difference and the actions that it needs to take in order to fulfill the desired state. So the plan shows that it will create 1 resource - 'dev-subnet-2' with all the attributes that are listed.

Now we confirm the plan -> yes

```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

aws_subnet.dev-subnet-2: Creating...
aws_subnet.dev-subnet-2: Creation complete after 1s [id=subnet-0b95d93084673c81b]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
→ terraform git:(Module_12/Terraform_chapter-3) x █

```

It is done! Let's check in the UI

▼

Subnets

PC dashboard

▼

VS Global View

▼

Filter by VPC:

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

▼

RECAP

Consistent syntax

```

12 vpc_id = aws_vpc.development-vpc_id
13
14 availability_zone = eu-central-1a
15 }

```

Configuration syntax is the same for all Providers

Provider -> is like import library

Resource / data -> is like a function call of a library

- resource -> function that creates something
- data -> function that returns something that already exists

Arguments -> are like the parameters of a function


```

1 provider "aws" {
2   region = "eu-central-1"
3   access_key = "AKIAUZ7NN5BBID2IQDHY"
4   secret_key = "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"
5 }
6
7 resource "aws_vpc" "development-vpc" {
8   cidr_block = "10.0.0.0/16"
9 }
10
11 resource "aws_subnet" "dev-subnet-1" {
12   vpc_id = aws_vpc.development-vpc.id
13   cidr_block = "10.0.10.0/24"
14   availability_zone = "eu-central-1a"
15 }
16
17 data "aws_vpc" "existing_vpc" {
18   default = true
19 }
20
21 resource "aws_subnet" "dev-subnet-2" {
22   vpc_id = data.aws_vpc.existing_vpc.id
23   cidr_block = "172.31.48.0/20"
24   availability_zone = "eu-central-1a"
25 }

```

Programming

provider =
import library

resource/data =
function call of library

arguments =
parameters of function

We need credentials:

```

2   region = "eu-central-1"
3   access_key = "AKIAUZ7NN5BBID2IQDHY"
4   secret_key = "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"
5 }
6
7 resource "aws_vpc" "development-vpc" {
8   cidr_block = "10.0.0.0/16"
9 }
10
11 resource "aws_subnet" "dev-subnet-1" {
12   vpc_id = aws_vpc.development-vpc.id
13   cidr_block = "10.0.10.0/24"
14   availability_zone = "eu-central-1a"
15 }
16
17 data "aws_vpc" "existing_vpc" {
18   default = true

```



**AWS User needs
necessary permissions!**

What happens if I execute
-> terraform apply

```

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.
➔ terraform git:(Module_12/Terraform_chapter-3) x terraform apply
data.aws_vpc.existing_vpc: Reading...
aws_vpc.development-vpc: Refreshing state... [id=vpc-0e43e232c4a94ae2a]
data.aws_vpc.existing_vpc: Read complete after 0s [id=vpc-03677c88f7cc2f715]
aws_subnet.dev-subnet-2: Refreshing state... [id=subnet-0b95d93084673c81b]
aws_subnet.dev-subnet-1: Refreshing state... [id=subnet-0ed81ef156fc90fa9]

No changes. Your infrastructure matches the configuration.

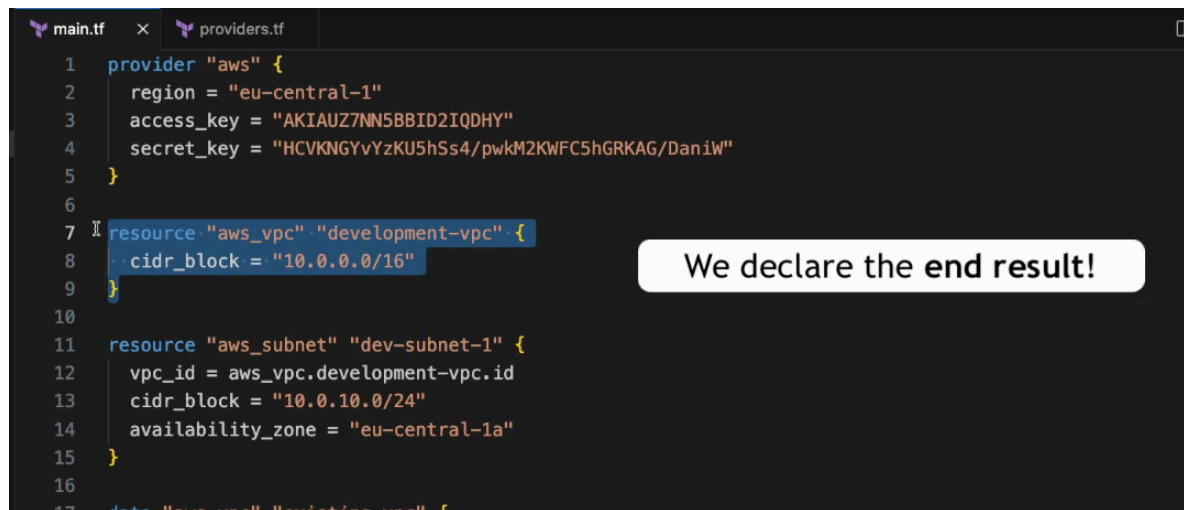
Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
➔ terraform git:(Module_12/Terraform_chapter-3) x

```

Tf refreshed the state for each resource and figured out that no changes are need it.

TF declare the end result, we didn't say how

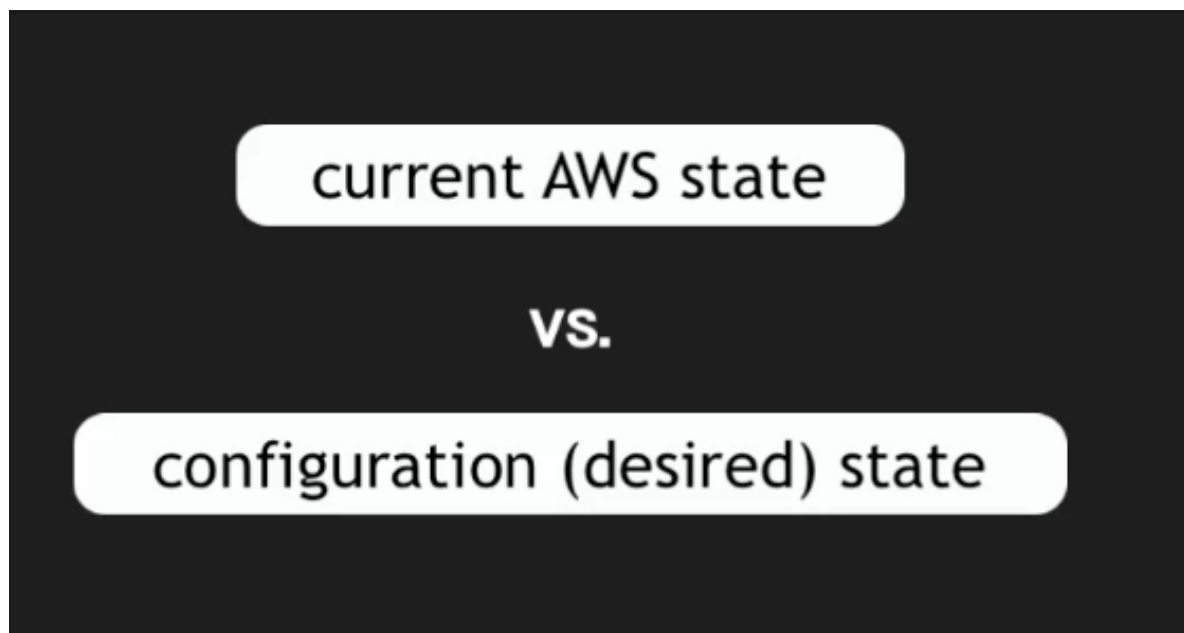


```
1 provider "aws" {
2   region = "eu-central-1"
3   access_key = "AKIAUZ7NN5BBID2IQDHY"
4   secret_key = "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"
5 }
6
7 resource "aws_vpc" "development-vpc" {
8   cidr_block = "10.0.0.0/16"
9 }
10
11 resource "aws_subnet" "dev-subnet-1" {
12   vpc_id = aws_vpc.development-vpc.id
13   cidr_block = "10.0.10.0/24"
14   availability_zone = "eu-central-1a"
15 }
16
17 data "aws_vpc" "existing-vpc" {
```

We declare the end result!

Then when we -> terraform apply

TF figures out the current AWS state and compares it with the desired state



If the states differed then TF creates whatever needs to be created.

This gives TF idempotency -> if we apply the TF config 100 times, then 100 times the result is the same.

Another advantage is that the user doesn't need to remember the current state.

main.tf × providers.tf

```
1 provider "aws" {
2   region = "eu-central-1"
3   access_key = "AKIAUZ7NN5BBID2IQDHY"
4   secret_key = "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"
5 }
6
7 resource "aws_vpc" "development-vpc" {
8   cidr_block = "10.0.0.0/16"
9 }
10
11 resource "aws_subnet" "dev-subnet-1" {
12   vpc_id = aws_vpc.development-vpc.id
13   cidr_block = "10.0.10.0/24"
14   availability_zone = "eu-central-1a"
15 }
16
17 data "aws_vpc" "existing_vpc" {
18   default = true
19 }
20
21 resource "aws_subnet" "dev-subnet-2" {
22   vpc_id = data.aws_vpc.existing_vpc.id
23   cidr_block = "172.31.48.0/20"
24   availability_zone = "eu-central-1a"
25 }
```



Idempotent

No need to remember the current state

